Geometry and Dynamics of LayerNorm

Paul M. Riechers[1, 2, *]

[1] *Simplex AI Safety, Berkeley, CA*

[2] *Beyond Institute for Theoretical Science (BITS), San Francisco, CA*

(Dated: May 8, 2024)

A technical note aiming to offer deeper intuition for the LayerNorm function common in deep neural networks. LayerNorm is defined relative to a distinguished 'neural' basis, but it does more than just normalize the corresponding vector elements. Rather, it implements a composition—of linear projection, nonlinear scaling, and then affine transformation—on input activation vectors. We develop both a new mathematical expression and geometric intuition, to make the net effect more transparent. We emphasize that, when LayerNorm acts on an N-dimensional vector space, all outcomes of LayerNorm lie within the intersection of an $(N-1)$-dimensional hyperplane and the interior of an N-dimensional hyperellipsoid. This intersection is the interior of an $(N-1)$-dimensional hyperellipsoid, and typical inputs are mapped near its surface. We find the direction and length of the principal axes of this $(N-1)$-dimensional hyperellipsoid via the eigen-decomposition of a simply constructed matrix.

arXiv:2405.04134v1 [cs.LG] 7 May 2024

INTRODUCTION

LayerNorm is a relatively simple function and so is often taken for granted. But, as one of the few building blocks of modern transformers (and other deep neural networks), it is worth understanding deeply. At the risk of stating things that are obvious, this note aims to very explicitly think through the LayerNorm function, how it maps activations, and the resultant geometry of activations.

LayerNorm acts on each N-dimensional activation vector $\vec{a} \in \mathbb{R}^N$ via a nonlinear transformation induced by learned parameters $\vec{g}$, $\vec{b} \in \mathbb{R}^N$ and fixed small parameter $\epsilon \in \mathbb{R}_{\geq 0}$ [1, 2]. Let $\vec{1}$ denote the N-dimensional vector of all ones in the neural basis. Then LayerNorm can be written as

$$\text{LayerNorm}(\vec{a}, \vec{g}, \vec{b}, \epsilon) = \vec{g} \odot \frac{(\vec{a} - \mu\vec{1})}{\sqrt{\sigma^2 + \epsilon}} + \vec{b} \,. \tag{1}$$

where $\odot$ denotes the element-wise product (i.e., Hadamard product), the mean neural activation is

$$\mu = \vec{a} \cdot \vec{1}/N \tag{2}$$

and the variance of activations across neurons is

$$\sigma^2 = (\vec{a} - \mu\vec{1}) \cdot (\vec{a} - \mu\vec{1})/N \,. \tag{3}$$

Typically, the gain $\vec{g}$ and bias $\vec{b}$ are learned during training. At the beginning of training, $\vec{g} = \vec{1}$ and $\vec{b} = \vec{0}$, and $\epsilon = 10^{-5}$ by default. This can alternatively be written in standard matrix-multiplication notation as

$$\text{LayerNorm}(\vec{a}, \vec{g}, \vec{b}, \epsilon) = \text{diag}(\vec{g}) \frac{(\vec{a} - \mu\vec{1})}{\sqrt{\sigma^2 + \epsilon}} + \vec{b} \,, \tag{4}$$

where $\text{diag}(\vec{g})$ is the matrix of all zeros except for the elements of $\vec{g}$ along its diagonal. At the beginning of training, $\text{diag}(\vec{g})$ is the identity matrix, I.

DECOMPOSING AND VISUALIZING LAYERNORM

LayerNorm's net action on the N-dimensional activation vector $\vec{a}$ can be thought of as a composition of several other simple functions (i)-(iv), as depicted in Fig. 1. First, LayerNorm performs a linear projection:

(i) Find the mean value of components in the neural basis, and subtract this mean from each component. Notably $\vec{a}' = \vec{a} - \mu\vec{1} = \vec{a} - (\vec{a} \cdot \hat{1})\hat{1} = (I - \hat{1}\hat{1}^\top)\vec{a}$ can be seen as the projection of $\vec{a}$ onto the $(N-1)$-dimensional hyperplane perpendicular to $\vec{1}$.

Here, $\hat{1} = \vec{1}/\sqrt{N}$ is the unit vector in the direction of $\vec{1}$. Note that $\vec{a}' \cdot \hat{1} = \vec{0}$ since $\vec{a}' \cdot \hat{1} = \left[\vec{a} - (\vec{a} \cdot \hat{1})\hat{1}\right] \cdot \hat{1} = \vec{a} \cdot \hat{1} - (\vec{a} \cdot \hat{1})\hat{1} \cdot \hat{1} = \vec{a} \cdot \hat{1} - \vec{a} \cdot \hat{1} = \vec{0}$. As a consequence, this initial sub-step of LayerNorm projects the activation vector onto a hyperplane that enforces $\sum_n a'_n = 0$, where a'_n is the nth component of the resultant vector $\vec{a}'$ in the neural basis.

Let's denote the rank-$(N-1)$ projector $\Pi = I - \hat{1}\hat{1}^\top$. Then we can write the result of sub-step (i) simply as $\vec{a}' = \Pi\vec{a}$. Note that, as a projector, $\Pi^2 = \Pi$. Hence, the variance can either be expressed as $\sigma^2 = \vec{a}^\top \Pi^\top \Pi \vec{a}/N = \vec{a}^\top \Pi \vec{a}/N$, or in relation to the square magnitude of $\vec{a}'$ since $\sigma^2 = |\Pi\vec{a}|^2/N$.

Subsequently, LayerNorm performs a nonlinear scaling:

(ii) Scale the magnitude of the resultant vector by $(\sigma^2 + \epsilon)^{-1/2}$, after which it is no larger than $\sqrt{N}$. This

scales the vector to bring it within the N-ball. If ϵ is small, then resultant points will be concentrated towards a magnitude of $\sqrt{N}$.

After the first two sub-steps (i) and (ii), the activations have been projected and nonlinearly scaled according to:

$$\vec{a} \mapsto \begin{cases} \vec{0} & \text{if } \vec{a} \propto \vec{1} \\ \frac{\sqrt{N}\,\widehat{\Pi\vec{a}}}{\sqrt{1+N\epsilon/\vec{a}^\top\Pi\vec{a}}} & \text{otherwise} \end{cases} . \tag{5}$$

$\widehat{\Pi\vec{a}}$ is a unit vector in the direction of $\Pi\vec{a}$, while the denominator $\sqrt{1+N\epsilon/\vec{a}^\top\Pi\vec{a}}$ is never smaller than 1. Accordingly, after these first two sub-steps, every input activation vector is mapped into the intersection of (a) the $(N-1)$-dimensional hyperplane perpendicular to $\vec{1}$ and (b) the N-ball of radius $N^{1/2}$.

After these first two sub-steps, the N-sphere and the points it now contains are subsequently (iii) stretched by $\vec{g}$ along the neural-basis directions into a hyperellipsoid, and then (iv) shifted by $\vec{b}$. Together, sub-steps (iii) and (iv) constitute an affine transformation. The linear transformation (iii)—matrix multiplication by $\mathrm{diag}(\vec{g})$—is diagonal in the neural basis, and maps the open N-ball containing all points to the interior of an N-dimensional hyperellipsoid with principal axes given by the neural basis. Likewise, it stretches and so tilts the intersecting hyperplane that contains all relevant points. The bias (iv) simply shifts the entire hyperellipsoid by $\vec{b}$.

We can now combine all these sub-steps to re-express LayerNorm as

$$\mathrm{LayerNorm}(\vec{a},\vec{g},\vec{b},\epsilon) = \sqrt{N}\mathrm{diag}(\vec{g})\,\frac{\Pi\vec{a}}{\sqrt{|\Pi\vec{a}|^2+N\epsilon}} + \vec{b}\ , \tag{6}$$

where we have defined the projector $\Pi = I - \hat{1}\hat{1}^\top$. This expression has the advantage of making all $\vec{a}$-dependence explicit, while highlighting the projection, scaling, and subsequent affine transformation.

The sequence of sub-steps building up the dynamics and geometry of LayerNorm is depicted in Fig. 1. Although LayerNorm is usually used in high-dimensional vector spaces, it is instructive to visualize and understand how it would operate in three dimensions, as shown, from which higher-dimensional behavior can be both intuitively and rigorously extrapolated.

In particular, when LayerNorm acts on an N-dimensional vector space, all outcomes of LayerNorm lie within the intersection of an $(N-1)$-dimensional hyperplane and an N-dimensional hyperellipsoid. This intersection is itself a hyperellipsoid, now of dimension $N-1$. After a sprinkling of points is transformed by LayerNorm, most of the points subsequently contained in this $(N-1)$-dimensional hyperellipsoid are concentrated towards its surface, as shown in Fig. 2.

Orthogonal subspace

If we subtract the bias, then the image of all activations after LayerNorm is orthogonal to at least one dimension of the vector space. This orthogonal space $\mathcal{N}$ can be found as the left nullspace of $\mathrm{diag}(\vec{g}) - \vec{g}\vec{1}^\top/N$. Let $\{|n\rangle\}_{n=1}^N$ denote the orthonormal neural basis. If the elements of $\vec{g}$ are all non-zero in the neural basis—i.e., if $g_n = \vec{g}^\top|n\rangle \neq 0$ for any n—then we find that the orthogonal subspace is the one-dimensional subspace of all vectors proportional to the inverse elements of $\vec{g}$. I.e., $\mathcal{N} = \{c\vec{\alpha} : c \in \mathbb{R}, \text{ and } \alpha_n = \vec{\alpha}^\top|n\rangle = 1/g_n\}$. However, if $g_n = 0$ for at least one n, then the number of orthogonal directions is equal to the number of these elements equal to zero, with the orthogonal subspace now given instead by $\mathcal{N} = \mathrm{span}\big(\{|n\rangle : g_n = 0\}\big)$.

Principal axes

The neural basis forms the principal axes of the N-dimensional hyperellipsoid after the N-sphere is stretched by $\vec{g}$. But what are the principal axes of the cross-sectional $(N-1)$-hyperellipsoid induced by $\vec{g}$? This is a rather more difficult question.

We find that the principal axes are the eigenstates of $\Pi_2 G^{-2}\Pi_2$, with semi-axes of length $\sqrt{N/\lambda}$ where λ is the eigenvalue associated with the eigenstate, $G = \mathrm{diag}(\vec{g})$, and $\Pi_2 = I - \vec{\alpha}\vec{\alpha}^\top/\vec{\alpha}^\top\vec{\alpha}$. As above, $\vec{\alpha}$ contains the reciprocal elements of the scaling vector in the neural basis: $\alpha_n = \vec{\alpha}^\top|n\rangle = 1/g_n$.

If $\vec{g}$ has neural-basis components equal to zero, i.e., if $g_n = 0$ for some n, then G^{-2} should be interpreted as the square of G's Drazin inverse [3], while we can then set either $\Pi_2 = I$ or $\Pi_2 = I - |n\rangle\langle n|$, with $\langle n|m\rangle = \delta_{n,m}$. If $g_n \neq 0$ for any n, then $\Pi_2 G^{-2}\Pi_2$ has a single zero eigenvalue, with corresponding eigenstate $\vec{\alpha}$ in the subspace orthogonal to LayerNorm's image.

Lengths of the semi-axes can alternatively be found as $\sqrt{N\zeta}$ where each ζ is an eigenvalue of $\Pi G^2\Pi$. However, unlike the use of $\Pi_2 G^{-2}\Pi_2$ above, this does not deliver the directions of the principal axes.

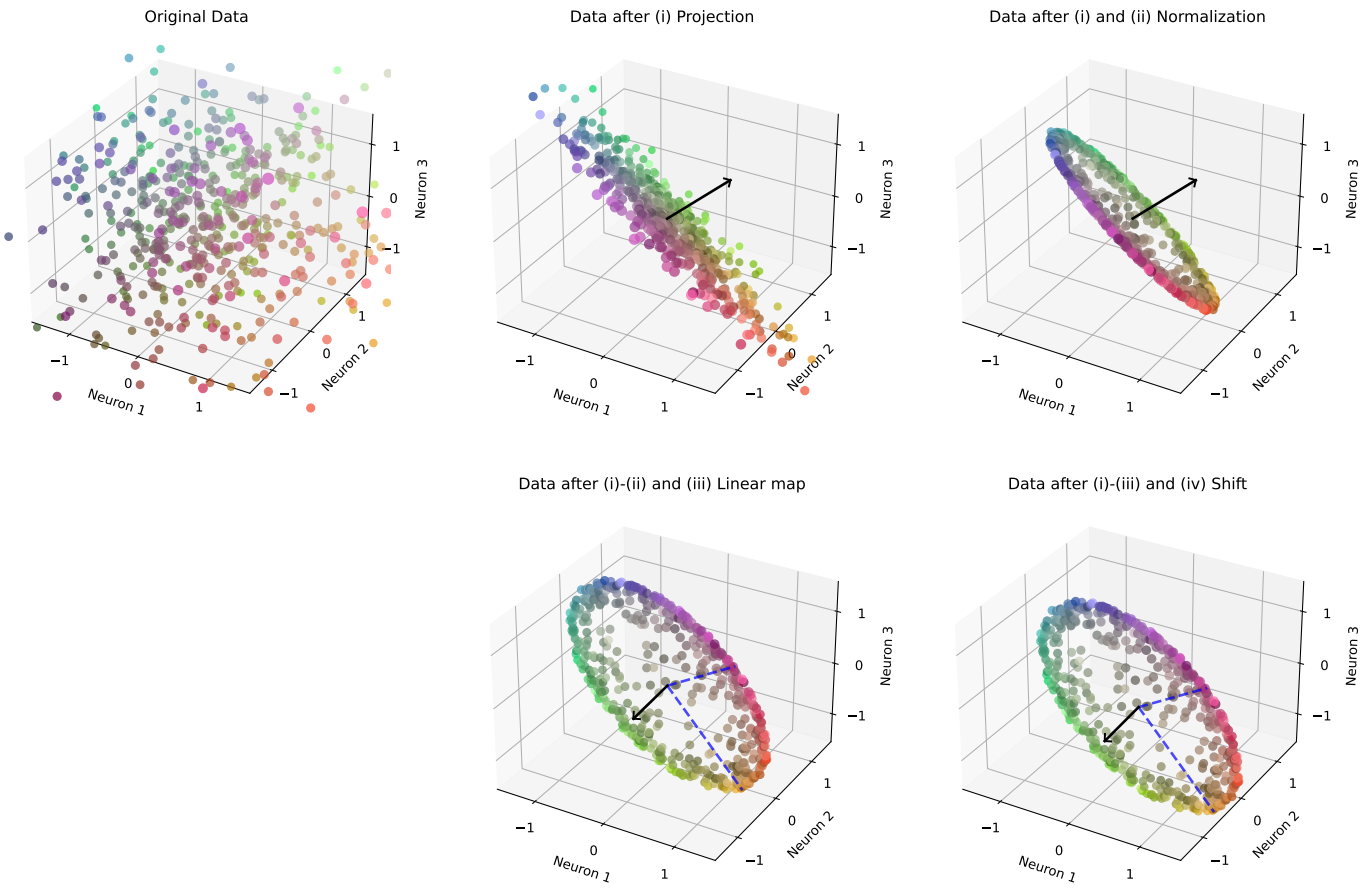

FIG. 1. LayerNorm as a composition of (i) Projection, (ii) Normalization, (iii) Linear transformation, and (iv) Global shift. Each point in the sequence of panels represents an activation vector input to LayerNorm. The (R,G,B) color values of each point directly encode the original activation of input neurons 1, 2, and 3, respectively in this simple example with $N = 3$. The second and third panels explicitly show the $\hat{1}$ vector orthogonal to the initial projection, while the last two panels show the unit vector $\hat{\alpha} = \vec{\alpha}/\alpha$ (with neural-basis components of $\vec{\alpha}$ given by $\alpha_n = 1/g_n$) which is orthogonal to the new plane after the linear transformation by $\mathrm{diag}(\vec{g})$. (The arrow depicting $\hat{\alpha}$ is shifted by $\vec{b}$ in the final case.) The last two panels also show the semi-axes of the principal axes as dashed blue lines, which correspond to eigenstates of $\Pi_2 G^{-2} \Pi_2$, with length $\sqrt{N/\lambda}$. We have depicted an unusually large $\epsilon = 1/10$ to demonstrate that the interior of the hyperellipsoid is not strictly empty after LayerNorm. But see Fig. 2.

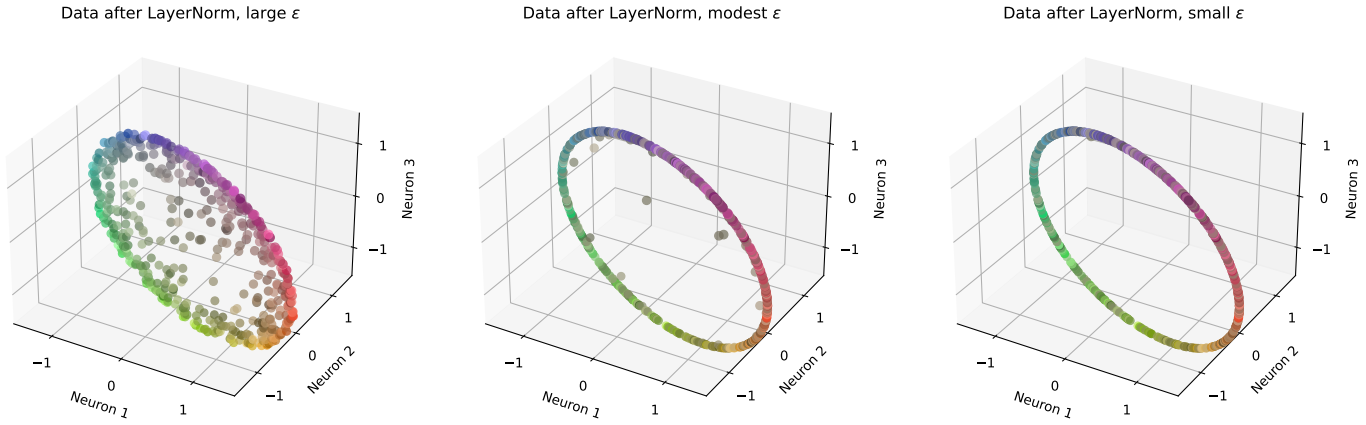

FIG. 2. From left to right, we compare the net effect of LayerNorm for $\epsilon = 10^{-1}$, 10^{-3}, and 10^{-5}. The final case corresponds to the default value for this small parameter in the standard PyTorch LayerNorm function. In practice, we should expect that most points get mapped very near the surface of the hyperellipsoid.